

Support Vector Regression (SVR)

Importing the libraries

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
In [2]: dataset = pd.read_csv(r"C:\Users\sande\OneDrive\Desktop\Naresh IT\data science and dataset")
```

```
Out[2]:
```

	Position	Level	Salary
0	Jr Software Engineer	1	45000
1	Sr Software Engineer	2	50000
2	Team Lead	3	60000
3	Manager	4	80000
4	Sr manager	5	110000
5	Region Manager	6	150000
6	AVP	7	200000
7	VP	8	300000
8	CTO	9	500000
9	CEO	10	1000000

```
In [3]: X = dataset.iloc[:, 1:2].values
y = dataset.iloc[:, 2].values
```

Fitting SVR to the dataset

```
In [4]: from sklearn.svm import SVR
regressor = SVR(kernel = 'poly', degree = 4, gamma="auto")
regressor.fit(X, y)
```

Out[4]:

SVR		
Parameters		
kernel	'poly'	
degree	4	
gamma	'auto'	
coef0	0.0	
tol	0.001	
C	1.0	
epsilon	0.1	
shrinking	True	
cache_size	200	
verbose	False	
max_iter	-1	

```
In [5]: from sklearn.neighbors import KNeighborsRegressor
regressor_knn = KNeighborsRegressor(n_neighbors=4)
regressor_knn.fit(X,y)
```

Out[5]:

KNeighborsRegressor		
Parameters		
n_neighbors	4	
weights	'uniform'	
algorithm	'auto'	
leaf_size	30	
p	2	
metric	'minkowski'	
metric_params	None	
n_jobs	None	

```
In [6]: from sklearn.tree import DecisionTreeRegressor
regressor_dt = DecisionTreeRegressor(splitter="random", criterion='absolute_error')
regressor_dt.fit(X,y)
```

Out[6]:

DecisionTreeRegressor		
▼ Parameters		
	criterion	'absolute_error'
	splitter	'random'
	max_depth	None
	min_samples_split	2
	min_samples_leaf	1
	min_weight_fraction_leaf	0.0
	max_features	None
	random_state	None
	max_leaf_nodes	None
	min_impurity_decrease	0.0
	ccp_alpha	0.0
	monotonic_cst	None

```
In [7]: from sklearn.ensemble import RandomForestRegressor
reg_rf = RandomForestRegressor(random_state=0, n_estimators = 50)
reg_rf.fit(X,y)
```

Out[7]:

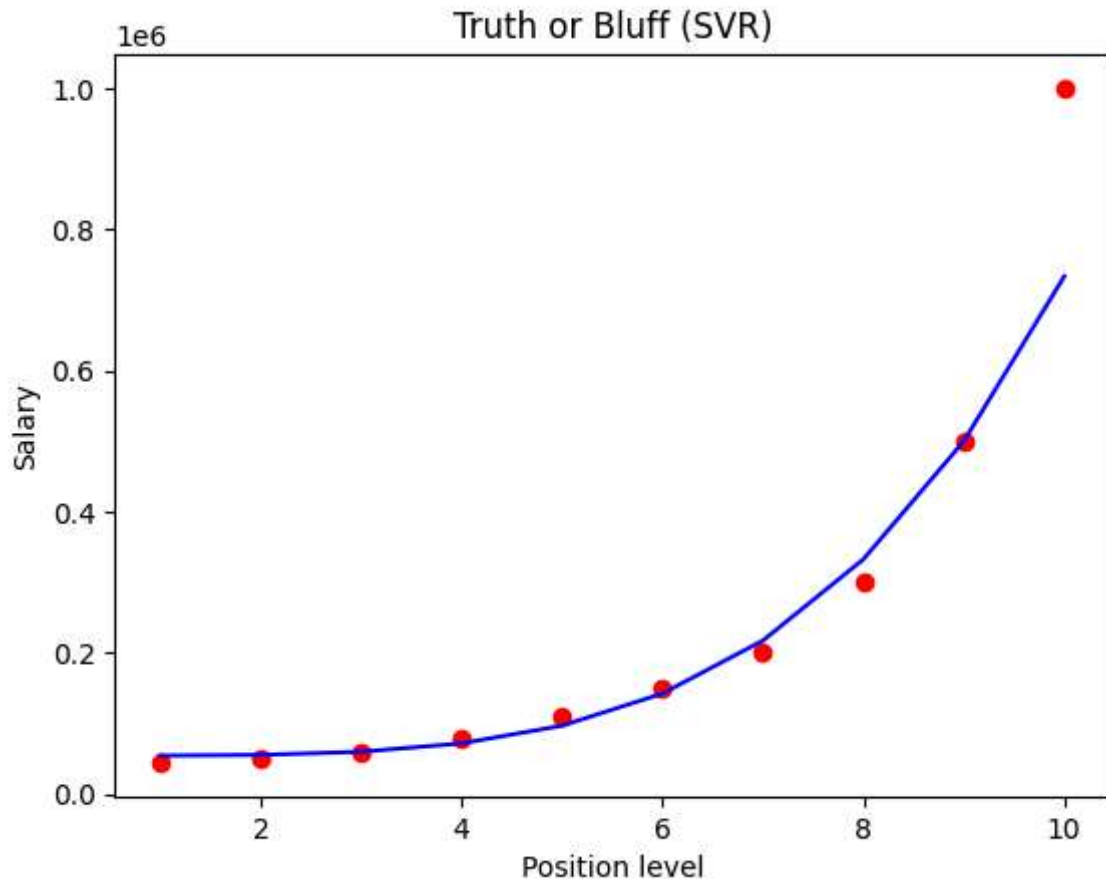
RandomForestRegressor		
Parameters		
	n_estimators	50
	criterion	'squared_error'
	max_depth	None
	min_samples_split	2
	min_samples_leaf	1
	min_weight_fraction_leaf	0.0
	max_features	1.0
	max_leaf_nodes	None
	min_impurity_decrease	0.0
	bootstrap	True
	oob_score	False
	n_jobs	None
	random_state	0
	verbose	0
	warm_start	False
	ccp_alpha	0.0
	max_samples	None
	monotonic_cst	None

```
y_pred_knn = regressor_knn.predict([[6.5]])
```

```
In [8]: y_pred_svr = regressor.predict([[6.5]])
```

Visualising the SVR results

```
In [9]: plt.scatter(X, y, color = 'red')
plt.plot(X, regressor.predict(X), color = 'blue')
plt.title('Truth or Bluff (SVR)')
plt.xlabel('Position level')
plt.ylabel('Salary')
plt.show()
```



Visualising the SVR results (for higher resolution and smoother curve)

```
In [10]: X_grid = np.arange(min(X), max(X), 0.01) # choice of 0.01 instead of 0.1 step becau
X_grid = X_grid.reshape((len(X_grid), 1))
plt.scatter(X, y, color = 'red')
plt.plot(X_grid, regressor.predict(X_grid), color = 'blue')
plt.title('Truth or Bluff (SVR)')
plt.xlabel('Position level')
plt.ylabel('Salary')
plt.show()
```

C:\Users\sande\AppData\Local\Temp\ipykernel_24848\3335682722.py:1: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)

X_grid = np.arange(min(X), max(X), 0.01) # choice of 0.01 instead of 0.1 step because the data is feature scaled

